Baxter Volkswagen: Jetta S Inventory Policy

Executive Summary

March 29, 2026

To whom it may concern:

Baxter Volkswagen in Omaha is reevaluating its inventory and ordering policy regarding the popular Jetta S model after a decrease in marginal profits was observed. The dealership currently follows a non-optimized $(14, 9)$ inventory policy, which does not maximize lot space and results in relatively frequent ordering with expected costs totaling \$34,071.03 per week. After developing a model to evaluate all feasible (q, r) policies, **our findings indicate that Baxter Volkswagen could be saving around \$29,484 per year by switching to a $(\mathbf{15}, \mathbf{5})$ policy**. Should immediate attention be given to this recommendation, Baxter could not only anticipate significant cost savings but also experience increased efficiency through less ordering and more emphasis on existing inventory.

Using the structure and assumptions of a Markov Chain, we developed a model that maps out on a week-by-week basis the fluctuations of inventory due to the combination of demand and ordering. For each (q, r) policy tested, long-run probabilities for all possible inventory levels are generated. These probabilities are each multiplied by their respective inventory level's weekly cost and added to achieve an average, balanced estimate of the policy cost on a weekly basis. In our analysis of the current $(14, 9)$ policy, it was observed inventory tends to stay relatively full due to a higher ordering threshold, potentially contributing to higher weekly costs.

In the effort to ensure our solution was accurate to the highest degree, we used a computer program to implement our model and test every feasible (q, r) policy. This involved testing every policy from $(0, 0)$ to $(15, 15)$, and all their expected costs are provided in tables for your convenience in this report. The most important of these, of course, is the minimal cost, found to be generated by the $(15, 5)$ policy. This policy makes full use of available lot space and only orders when necessary, taking care to avoid special orders. Valuable improvement in business operations and expected costs exists with this policy that saves around \$567 per week.

By taking full advantage of lower inventory costs, Baxter Volkswagen could see a turnaround in profits. **In fact, with the adoption of the $(\mathbf{15}, \mathbf{5})$ policy, Baxter's predicted weekly costs would be expected to decrease by about 1.66% to \$33,503.97**. Furthermore, we would like to propose that Baxter explore other sources for obtaining cars for inventory, such as offering the option to trade-in. More options for obtaining vehicles could significantly impact weekly costs by unlocking the possibility of avoiding the full costs of ordering. At Deloitte Consulting, we would be happy to provide our services should Baxter choose to investigate more possibilities for inventory replacement in the continual effort to decrease costs.

Thank you for considering our recommendations,

Nathan Schmitz and Natalie McNamara, on behalf of Deloitte Consulting

Table of Contents

Technical Report

1. Introduction

Technical Report

Baxter Auto is a car dealership conglomerate with locations in Nebraska, Kansas, and Colorado. Baxter is looking to re-evaluate an inventory policy within the Baxter Volkswagen dealership in Omaha as decreases in marginal profits have been observed over the last three years; the inventory policy in question centers around the Volkswagen Jetta S Model. The current policy was empirically determined 10 years ago: our team was tasked with evaluating it and developing a program to determine a more optimal policy that will result in a decrease in weekly costs. Further reading of this report will reveal that a more optimal policy not only results in lowered expected weekly costs but also has the added benefit of lowering the threshold for ordering. Less ordering will contribute to greater overall efficiency in the weekly operations of Baxter Volkswagen.

In this report, we will begin by outlining the inventory policy that Baxter Volkswagen utilizes in addition to the parameters that surround weekly decision-making. Important assumptions were granted to us by Baxter Volkswagen that allowed us to make use of Markov Chains in obtaining our results – these assumptions are outlined in Section 3 of this report. Results were obtained from a computer program, specifically utilizing the Python programming language. In our discussion of the program development process, we will explore the construction of a transition matrix and the calculation of the long-run probabilities in line with our usage of Markov Chains. Expected weekly costs rely on these long-run probabilities to give us a sense of which policy offers the minimal weekly cost. To ensure thoroughness and accuracy in our results, we tested every feasible and logical inventory policy – that is, all nonnegative inventory policies were checked (we did not attempt to check policies where the maximum number of cars that could be held or ordered up to was below zero, nor did we check policies where the threshold for ordering up was negative).

Analysis of results and recommendations are provided in the latter part of this report; discussion will center around minimal cost policy in comparison to the current inventory policy. Trends among the policy results will be explored for more well-rounded knowledge of the problem. Baxter Volkswagen should aim to maximize lot space but make relatively infrequent orders, only ordering up to 15 vehicles when inventory on hand is less than or equal to 5 vehicles. Special orders are to be avoided since they tend to rack up costs quickly. *We believe that, by switching to this optimal inventory policy, Baxter Volkswagen could anticipate saving around $567 per week in inventory and ordering costs, translating to about $29,484 per year*. The results, analysis, and recommendations presented in this report should be carefully considered should Baxter want to improve profit margins both in the immediate future and long term.

2. Description of Problem

Technical Report

Our team was provided with a description of the current parameters surrounding Baxter Volkswagen's inventory policy, which include average demand levels and their respective probabilities, maximum lot space, ordering costs, and inventory interest. The structure of the inventory policy is discussed below. Baxter is looking for an improvement in weekly costs associated with inventory and ordering, and an optimized ordering policy is key to achieving lowered costs. Please carefully review the following:

Orders for vehicles to stock inventory are made at the beginning of each week on Monday morning. Management has made clear that a so-called (q, r) inventory policy is used on a week-by-week basis to determine how many vehicles to order for the upcoming week.

Table 1: *(q, r) inventory policy structure*

Given that:	Number of vehicles ordered:
$n \leq r$	$q - n$
$n > r$	0

Table 1 demonstrates how the inventory and ordering policy works. "n" represents the current inventory level at the end of the business week on Sunday night. "q" denotes the maximum number of vehicles that the dealership can have on hand for the week. "r" represents a "threshold" – if the inventory n is equal to or below r, the inventory is replenished with vehicles up to value q. Otherwise, no orders are made for the week. It should be noted that n can become negative if demand occurs during the week without existing inventory. In this case, special orders for these unfulfilled vehicles must be made on Monday in addition to any regular inventory ordering.

The current (q, r) inventory policy followed by Baxter Volkswagen is $(14, 9)$. Thus, if the inventory level at the end of the business week (Wednesday through Sunday) is 9 vehicles or less, an order for 5 vehicles will be made on Monday morning.

Table 2: *Weekly demand levels*

Average demand level	Percentage of time for demand
0 vehicles	10%
1 vehicle	20%
2 vehicles	40%
3 vehicles	25%
6 vehicles	5%

Table 2 displays the possible vehicle demand levels that can occur during the week along with the probabilities that each of those demand levels will occur.

Table 3: *Costs per weekly inventory and ordering*

Order cost (per car)	Special-order cost (per car)	Fixed order cost (per order)	Penalty cost (per car)	Interest rate (per week)
$16000	$16000 + $3000	$3500	$20000	0.0175/52

Table 3 contains information regarding costs associated with inventory and ordering. Orders can occur at the beginning of each week (Monday morning) if the current inventory level and inventory policy necessitate an order. If an order is to be placed, a fixed cost of $3500 is incurred on top of the costs per car ordered as shown in Table 3. Each car that is regularly ordered costs $16000. Table 3 notes that there is a special-order cost, which is incurred per car that must be delivered to a customer's house due to lack of inventory on the lot at the time of the customer's purchase. A $20000 penalty cost also occurs for each car that must be specially ordered due to the potential loss of a customer or lost sales.

A yearly interest rate of 1.75% dictates the cost of inventory. Because the process of ordering (and any subsequent inventory fluctuations) occurs week-to-week, the interest rate has been scaled to a weekly rate in Table 3. This is necessary for expected cost calculations during the Markov Chain implementation process and will keep all parameters consistent. Thus, if n represents the inventory level on hand, and the value of each car is the same as the purchasing price of $16000, then the ***cost of inventory per week*** can be represented as follows:

$$\frac{0.0175}{52}(16000) * n$$

The maximum capacity of Baxter's lot is ***15 vehicles***.

Baxter Volkswagen is looking to determine:

1. The optimal threshold r for ordering
2. The optimal number q to order up to

Influenced by weekly demand, the combination of the end-of-week inventory level, the threshold for ordering, and the number of vehicles to order up to dictate the weekly expected costs. The current policy of $(14, 9)$ will be subject to analysis later in this report as we seek to find and justify a more optimal policy for Baxter to adopt.

3. Assumptions

Technical Report

Assumptions made during the development of our model generally fall into two categories: assumptions regarding the weekly operations of Baxter Volkswagen, and assumptions made for the implementation of a Markov Chain.

General assumptions are as follows:

- Any action (i.e. inventory evaluation and ordering) occurs on a week-to-week basis.
- Orders, if necessary, occur only once a week – no extra orders are placed during the business week. Cars placed on special order are included in the once-a-week Monday order.
- The inventory cost calculations assume that the value of the car on the lot is the same as its purchase price ($16000).
- All parameters given to us – prices, costs, interest rate, etc. – remain unchanged from week to week for the year.
- If no inventory exists but demand is present, the car can still be "sold" but is put on "backorder", resulting in temporary negative inventory that is resolved with the Monday order.

Two critical assumptions are made for the implementation of a Markov Chain:

- The demand from one week to another week is constant, which satisfies the time independence assumption needed for implementation of the Markov Chain. This assumption means that transition probabilities do not change over time.
- The demand from one week to another week is memoryless, a necessary property needed for implementation of the Markov Chain. This property ensures that past events have no bearing on future events. In this case of weekly demand, this means that any high or low demand from the previous week will not affect the demand for the following week.

4. Program Development and Implementation

Technical Report

This section details the development and implementation of the model. The expected cost values for every (q, r) policy with q values ranging from 0 to 15 and r values ranging from 0 to 15 are calculated. Policies with a negative r value were excluded due to the extra cost associated with special orders, and since each occurrence of negative inventory acquires large costs, policies that require negative inventory before ordering were removed as a possibility. Policies with a negative q value were excluded since it should be required to order up to an inventory of at least zero – ordering "up" to a negative value does not make logical sense in this case. The minimum of these policies is displayed, and the output of all policies is saved to an Excel (.xlsx) file.

This model was developed in Python with the libraries NumPy and Pandas. NumPy was used for matrix operations, and Pandas was used for its DataFrame capabilities so that matrices could have custom indices. In addition, the constants given by the problem description are defined at the start of the program. The imports of the libraries and definitions of the constants in Python are shown in Figure 1.

Figure 1: *Libraries and constants of the Python implementation*

```python
import numpy as np
import pandas as pd

# Constants
demand_probabilities = [(0, 0.1), (1, 0.2), (2, 0.4), (3, 0.25), (6, 0.05)] # List of tuples for the demand for x cars is probability y
sale_price = 20000
purchase_price = 16000
order_cost = 3500
inventory_rate = 0.0175 / 52 # Yearly cost of capital divided by weeks per year
special_order_cost = 23000 # $20000 penalty for loss in buisness plus the $3000 extra special order expences

```

For a given (q, r) policy '$M_{q,r}$', the expected cost for a given week depends on the number of cars in inventory and the long-run probability of having that amount of cars in inventory. Thus, the general form of the expected cost multiplies the long-run probability of having i cars in inventory before making the order on Monday times the expected cost of having inventory i. Figure 2 demonstrates the Python code for this logic.

$$E[M_{q,r}] = \sum_{i=r-6}^{q} P(i) * E(i)$$

Figure 2: *Calculation of expected cost for policy (q, r) in Python*

```python

        # Use the cost and probability in each state to get the total expected value for this q and r policy
        policy_matrix.loc[r, q] = 0
        for i in range(r-6, q+1):
            policy_matrix.loc[r, q] += expected_cost[i] * state_probabilities[i]

```

In the Markov Chain construction process, a specific variable should be defined so we are informed as to exactly what the meaning of the indices on the edges of the transition matrices is:

$$X_t = number\ of\ cars\ in\ inventory\ on\ Sunday\ night\ (before\ any\ order)\ at\ week\ t$$

Thus, each state represents the inventory of Jetta S models after all cars have been sold during the week, but before the new order on Monday. This means that the state space could range anywhere from $r - 6$ to q depending on the policy. Generally, a policy cannot reach an inventory of $r - 6$, however; in the case where q is greater than or equal to r, it is possible to have a week with six cars in demand at an inventory of r, resulting in an inventory of $r - 6$. In cases where r is less than q, if the current number of cars is equal to r, an order is placed, so it is impossible to have an inventory of $r - 6$, and the probability will be zero. The maximum cars held is q since the only way to increase inventory is through an order and after an order, there are exactly q cars in inventory.

To calculate the probability of having inventory i at the end of the business week, a transition matrix must be built. The matrix designates previous week's (week $t - 1$) inventory (the current inventory on hand before any orders) as the rows and the upcoming week's (week t) inventory as the columns. This means that each row could have no cars sold with a probability of 0.1, one car sold with a probability of 0.2, two with a 0.4 probability, three with a probability of 0.25, and six with a probability of 0.05. There are two cases for the locations of these probabilities in each row, which depends on the (q, r) policy. If the previous week's inventory i was less than or equal to the ordering threshold r, then each probability will be inserted in column q minus the demand they are associated with, since an order will replenish inventory up to q. In the case that the previous week's inventory is greater than r, no inventory replenishing occurs, so the probabilities are inserted in column i (current inventory) minus potential cars sold. Figure 3 shows the Python implementation of this process.

Figure 3: *Creation of the transition matrix in Python*

```python

# Update the transition matrix rows and calculate costs per state
for i in range(r-6, q+1):
```

```python

    for cars_sold, prob in demand_probabilities:
        # The state we are in is less than r, so we ordering up to q then selling cars
        if i <= r:
            transition_matrix.loc[i, q - cars_sold] += prob

        # The state we are in is greater than r, so we are selling from the current inventory
        else:
            transition_matrix.loc[i, i - cars_sold] += prob

```

For instance, in the current $(14, 9)$ policy, if $i = 9 \leq r$, then probability 0.1 will be inserted in column $q = 14$ in current inventory row $i = 9$; an order was made to replenish inventory up to q for the upcoming week, and probability 0.1 is associated with selling no cars during the current week. In this same case, probability 0.2 is inserted in column $q = 13$ in current inventory row $i = 9$, since inventory was still replenished to 14 vehicles, but this probability is associated with selling one car during the week. However, if $i = 10 > r$, then probability 0.1 is simply inserted in column $i = 10$, since no inventory change will occur for the upcoming week with no order placed. The rest of the probabilities fall in line after this with respect to the demand they are associated with. The full transition matrix for the policy $(14, 9)$ is on display in Figure 4.

Figure 4: *Transition matrix for policy* $(14, 9)$

	3	4	5	6	7	8	9	10	11	12	13	14
3	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
4	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
5	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
6	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
7	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
8	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
9	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
10	0.0	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.00	0.0	0.0	0.0
11	0.0	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.0	0.0	0.0
12	0.0	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.1	0.0	0.0
13	0.0	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.2	0.1	0.0
14	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1

After computing the transition matrix, the steady state probabilities can be found with the following formula:

$$\pi_i = [(P - I)_{1:n-1} \mid 1]^{-1}{}_n$$

This formula takes the transition matrix P minus the identity matrix I. The last column is replaced with a series of ones, hence why the subtraction $P - I$ is only taken up to column $n - 1$. Finally, the inverse of this new matrix is taken, and the long-run probabilities are in the last row of the matrix. This process of computing the long-run probabilities in Python is shown in Figure 5. Notice that the identity matrix is customized to match the size of the transition matrix to carry out proper matrix calculations, which may be a different size each time depending on the policy. Matrix inverse calculations are carried out with use of the linear algebra library in NumPy.

Figure 5: *Calculation of probabilities in Python*

```python
    # Use the transition matrix to get the probability of being in each state
    identity = pd.DataFrame(data=np.identity(q-r+7), columns=range(r-6, q+1), index=range(r-6, q+1))
    s = transition_matrix - identity
    s.loc[r-6:q, q] = 1
    state_probabilities = pd.Series(np.linalg.inv(s)[-1], index=range(r-6, q+1))
```

For the current $(14, 9)$ policy, the long-run probabilities for each inventory state i are on display in Figure 6. For instance, the probability of having 8 cars in inventory at any point in time (time independence assumption of Markov Chains) is about 0.126. Notice that, for this case, the long-run probability of having 3 cars in inventory is 0.000. Referring to the transition matrix in Figure 4, notice that the policy restricts any chance of having 3 cars in inventory at any time, since we start ordering up right before any chance of having 3 cars can occur. We chose to incorporate the state of 3 vehicles in the matrix and long-run probabilities simply to show that r will restrict the chance of having 3 or less vehicles.

Figure 6: *Long-run probabilities for policy* $(14, 9)$

3	0.000000
4	0.007510
5	0.009382
6	0.009527
7	0.041838
8	0.126284
9	0.152733
10	0.150204
11	0.187638
12	0.190549
13	0.085747
14	0.038586

The last piece of information needed to calculate the expected cost of each (q, r) policy is the expected costs for inventory holdings and any orders that occur. This cost can be further broken down into the weekly order cost, weekly special-order cost, and inventory cost. These costs are given by the following formulas:

$$weekly\_order\_cost[i] = \begin{cases} 3500 + 16000(q - i) & if\ i \leq r\ and\ i \neq q \\ 0 & if\ i > r \end{cases}$$

$$weekly\_special\_order\_cost[i] = \begin{cases} -i(20000 + 3000) & if\ i < 0 \\ 0 & if\ i \geq 0 \end{cases}$$

$$inventory\_cost[i] = \begin{cases} 16000(i * \frac{0.0175}{52}) & if\ i > 0 \\ 0 & if\ i \leq 0 \end{cases}$$

$$E[i] = weekly\_order\_cost[i] + weekly\_special\_order\_cost[i] + inventory\_cost[i]$$

Weekly order costs only occur if i is equal to or below the ordering threshold r. This includes the fixed $3500 order cost in addition to the $16000 spent on each car that is ordered. Special order costs assume that inventory i has at some point during the week dipped below zero due to lack of inventory. This calculation essentially takes the absolute value of i by taking it times -1 to get the number of cars on "backorder"; then, this number is multiplied by the total cost of each special order, which includes the special-order cost of $3000 and the penalty cost of $20000. These special orders are then properly accounted for in the weekly order cost because q will subtract a negative number to get the total amount needed to be ordered, both special and regular. Then, the $16000 baseline cost is taken times this amount plus the fixed $3500. The inventory cost is only incurred when there is inventory on hand, which incorporates the value of the car and the interest rate. The conditions placed on each of these expressions allow for the full expected cost calculation $E[i]$ to be relatively straightforward, since each of the three costs will be held to zero if the combination of the policy and current inventory level call for it.

Figure 7 shows how the above information was translated into the program. Refer to Figure 1 for the constant values associated with the variables in each of these expressions. Notice that, in the weekly order cost calculation, we ensured that if i has already reached the maximum q, no order can be placed, even if i is still below the ordering threshold. This is pertinent to expected costs for policies where $q \leq r$ (see Section 5: Results and Analysis for more information on this). We needed to ensure that orders could not occur for these specific instances.

Figure 7: *Expected cost calculations for current inventory i in Python*

```python

        # Update the transition matrix rows and calculate costs per state
        for i in range(r-6, q+1):
            # The cost of a standard batch order when we have inventory i
            weekly_order_cost = order_cost + (q - i) * purchase_price if i <= r and i != q else 0

            # The cost of a special order when we have inventory i
            weekly_special_order_cost = special_order_cost * -i if i < 0 else 0

            # The cost of inventory when we have i cars stored
            inventory_cost = i * purchase_price * inventory_rate if i > 0 else 0

            # The total expected cost when we have i cars in inventory
            expected_cost[i] = weekly_order_cost  + weekly_special_order_cost + inventory_cost

```

The results of these cost calculations are stored in python dictionary. This dictionary for the costs of policy $(14, 9)$ is shown in Figure 8.

Figure 8: *Expected costs for each inventory amount for policy $(14, 9)$*

```
{3: 179516.15384615384, 4: 163521.53846153847, 5: 147526.92307692306, 6: 131532.3076923077, 7: 115537.69230769231, 8: 99543.07692307692, 9: 83548.46153846153, 10: 53.846153846153854, 11: 59.23076923076924, 12: 64.61538461538463, 13: 70.00000000000001, 14: 75.38461538461539}
```

The expected costs are calculated for every possible nonnegative (q, r) policy from 0 to 15 for q and 0 to 15 for r, in line with the maximum inventory holding space of 15. In Python, this is done through two nested for loops as shown in Figure 9. Each expected cost is then input into the policy matrix at its proper index.

Figure 9: *Iteration through all (q, r) policies in Python*

```python

# Get the expected value for each pair of q and r
policy_matrix = pd.DataFrame(columns=range(16), index=range(16))
for q in range(16):
    for r in range(q+1):

```

When all policies have been computed, the minimum cost policy is printed to the standard output of the program, and all results are saved to an Excel spreadsheet. The Python code to accomplish this is shown in Figure 10.

Figure 10: *Output of results in Python*

```python

#Display the minimum policy
min_q, min_r = policy_matrix.stack().idxmin()
print(f"Minimum cost policy occurs at q = {min_q}, r = {min_r}, cost = {policy_matrix.loc[min_q, min_r]:.2f}")

# Write the results to an excel file
policy_matrix.to_excel("Baxter_Auto_q_r_analysis_results.xlsx")

```

Please refer to Figure 14 in the Appendix for the full code unified in one picture used to implement our Markov Chain model.

5. Results and Analysis

Technical Report

The results of the program described in "Section 4: Program Development and Implementation" are presented in Tables 4, 5, and 6 below. An expected cost for each possible policy (q, r) is given, where q ranges from 0-15 and r ranges from 0-15. Values of q can be found on the left-hand side column of the table designating rows, while the values of r can be found along the top edge of the table designating columns.

The expected cost of the current policy $q = 14$ and $r = 9$ is $34,071.03 per week.

The minimum expected cost policy occurs at $q = 15$ and $r = 5$ at $33,503.97 per week.

Thus, switching policies will save Baxter Volkswagen $567.06 per week, or $29,487.12 per year. **You will find the expected costs of both the current policy and the optimal policy in Table 5 specifically (see below)**. Again, the tables below display all the expected costs for every policy, and only nonnegative logical policies were tested.

Table 4: *Expected costs per week for r values 0 through 4*

q \ r	0	1	2	3	4
0	$ 83,100.00	$ 83,100.00	$ 83,100.00	$ 83,100.00	$ 83,100.00
1	$ 62,400.54	$ 62,400.54	$ 62,400.54	$ 62,400.54	$ 62,400.54
2	$ 48,656.41	$ 46,302.15	$ 46,302.15	$ 46,302.15	$ 46,302.15
3	$ 45,601.73	$ 40,087.06	$ 39,405.92	$ 39,405.92	$ 39,405.92
4	$ 43,790.00	$ 39,408.90	$ 37,896.47	$ 38,261.04	$ 38,261.04
5	$ 41,237.57	$ 38,053.71	$ 36,608.95	$ 36,751.59	$ 37,116.15
6	$ 40,029.34	$ 36,767.43	$ 35,638.91	$ 35,464.07	$ 35,606.70
7	$ 39,380.13	$ 36,355.87	$ 35,202.52	$ 35,016.32	$ 34,989.48
8	$ 38,432.51	$ 35,938.28	$ 34,840.34	$ 34,600.11	$ 34,509.80
9	$ 37,951.67	$ 35,513.96	$ 34,598.82	$ 34,346.61	$ 34,216.88
10	$ 37,498.97	$ 35,300.10	$ 34,401.41	$ 34,179.22	$ 34,037.40
11	$ 37,074.61	$ 35,065.90	$ 34,248.46	$ 34,018.70	$ 33,887.82
12	$ 36,765.16	$ 34,869.73	$ 34,119.50	$ 33,903.53	$ 33,765.33
13	$ 36,487.05	$ 34,725.08	$ 34,014.71	$ 33,811.70	$ 33,680.35
14	$ 36,236.13	$ 34,588.62	$ 33,925.77	$ 33,729.35	$ 33,604.20
15	$ 36,030.91	$ 34,471.89	$ 33,850.50	$ 33,664.32	$ 33,542.36

Table 5*: *Expected costs per week for r values 5 through 9*

q \ r	5	6	7	8	9
0	$ 83,100.00	$ 83,100.00	$ 83,100.00	$ 83,100.00	$ 83,100.00
1	$ 62,400.54	$ 62,400.54	$ 62,400.54	$ 62,400.54	$ 62,400.54
2	$ 46,302.15	$ 46,302.15	$ 46,302.15	$ 46,302.15	$ 46,302.15
3	$ 39,405.92	$ 39,405.92	$ 39,405.92	$ 39,405.92	$ 39,405.92
4	$ 38,261.04	$ 38,261.04	$ 38,261.04	$ 38,261.04	$ 38,261.04
5	$ 37,116.15	$ 37,116.15	$ 37,116.15	$ 37,116.15	$ 37,116.15
6	$ 35,971.27	$ 35,971.27	$ 35,971.27	$ 35,971.27	$ 35,971.27
7	$ 35,402.95	$ 35,976.65	$ 35,976.65	$ 35,976.65	$ 35,976.65
8	$ 34,663.85	$ 35,408.33	$ 35,982.04	$ 35,982.04	$ 35,982.04
9	$ 34,261.20	$ 34,669.24	$ 35,413.72	$ 35,987.42	$ 35,987.42
10	$ 34,049.49	$ 34,266.58	$ 34,674.62	$ 35,419.10	$ 35,992.81
11	$ 33,875.90	$ 34,054.88	$ 34,271.97	$ 34,680.01	$ 35,424.49
12	$ 33,741.26	$ 33,881.29	$ 34,060.26	$ 34,277.35	$ 34,685.39
13	$ 33,647.64	$ 33,746.65	$ 33,886.67	$ 34,065.65	$ 34,282.74
14	$ 33,567.61	$ 33,653.03	$ 33,752.03	$ 33,892.06	$ 34,071.03
15	$ 33,503.97	$ 33,572.99	$ 33,658.41	$ 33,757.42	$ 33,897.44

*The red box indicates the expected cost of the current policy $(14, 9)$, while the yellow box indicates the expected cost of the recommended minimum cost policy $(15, 5)$.

Table 6: *Expected costs per week for r values 10 through 15*

q \ r	10	11	12	13	14	15
0	$ 83,100.00	$ 83,100.00	$ 83,100.00	$ 83,100.00	$ 83,100.00	$ 83,100.00
1	$ 62,400.54	$ 62,400.54	$ 62,400.54	$ 62,400.54	$ 62,400.54	$ 62,400.54
2	$ 46,302.15	$ 46,302.15	$ 46,302.15	$ 46,302.15	$ 46,302.15	$ 46,302.15
3	$ 39,405.92	$ 39,405.92	$ 39,405.92	$ 39,405.92	$ 39,405.92	$ 39,405.92
4	$ 38,261.04	$ 38,261.04	$ 38,261.04	$ 38,261.04	$ 38,261.04	$ 38,261.04
5	$ 37,116.15	$ 37,116.15	$ 37,116.15	$ 37,116.15	$ 37,116.15	$ 37,116.15
6	$ 35,971.27	$ 35,971.27	$ 35,971.27	$ 35,971.27	$ 35,971.27	$ 35,971.27
7	$ 35,976.65	$ 35,976.65	$ 35,976.65	$ 35,976.65	$ 35,976.65	$ 35,976.65
8	$ 35,982.04	$ 35,982.04	$ 35,982.04	$ 35,982.04	$ 35,982.04	$ 35,982.04
9	$ 35,987.42	$ 35,987.42	$ 35,987.42	$ 35,987.42	$ 35,987.42	$ 35,987.42
10	$ 35,992.81	$ 35,992.81	$ 35,992.81	$ 35,992.81	$ 35,992.81	$ 35,992.81
11	$ 35,998.19	$ 35,998.19	$ 35,998.19	$ 35,998.19	$ 35,998.19	$ 35,998.19
12	$ 35,429.87	$ 36,003.58	$ 36,003.58	$ 36,003.58	$ 36,003.58	$ 36,003.58
13	$ 34,690.78	$ 35,435.26	$ 36,008.96	$ 36,008.96	$ 36,008.96	$ 36,008.96
14	$ 34,288.12	$ 34,696.16	$ 35,440.64	$ 36,014.35	$ 36,014.35	$ 36,014.35
15	$ 34,076.42	$ 34,293.51	$ 34,701.55	$ 35,446.02	$ 36,019.73	$ 36,019.73

Note that many policies with the same q value but increasing r values all generate the same expected cost. For purposes of thoroughness, we listed costs for policies where $q \leq r$; since r is always at least equal to the maximum allowed inventory, orders will simply always occur for every possible current inventory value i (other than when $i = q$). Thus, the transition matrix and probabilities will be the same for all these policies with q in common, resulting in the same expected costs. A diagonal forms at policies where $r = q - 1$, shaded in gray in the tables above; to the right of this diagonal, the costs are the same in each row.

In general, the smaller the q, the higher the expected cost (no matter what the threshold r is). This is because the more q is limited, the higher the chance there is that a special order will need to occur during the week. A more noticeable expected cost increase begins around the maximum inventory value of $q = 5$. This is to be expected given that the maximum demand during the week can be up to 6 cars. If the maximum weekly inventory q is capped at 5 vehicles or lower, the chances that Baxter runs out of inventory during the week begin to grow. Thus, the trend of special orders driving up costs is revealed.

On a similar note, depending on the value of q, higher values of r tend to result in slightly higher expected costs. This is also to be expected given that higher values of r may result in more frequent ordering. However, r still needs to be high enough to initiate ordering before backorders start occurring. This reveals why the value of r in the optimal policy is 5 – as noted previously, when inventory is 5 vehicles or lower, the chance of backorders is introduced. Thus, to avoid this, r is set to the lowest ordering threshold it can be without introducing backorders ($r = 5$), resulting in our optimal policy.

The optimal minimum cost policy $(15, 5)$ is the subject of further analysis. The transition matrix for this policy is presented in Figure 11 below. Notice that the threshold r prevents the chance of the inventory for the upcoming week becoming "negative", as evidenced by the return of probabilities to the very right side of the matrix starting in row $i = 5$.

Figure 11: *Transition matrix for policy* $(15, 5)$

	-1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-1	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
0	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
1	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
2	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
3	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
4	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
5	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1
6	0.0	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.00	0.00	0.00	0.00	0.00	0.00	0.0	0.0	0.0
7	0.0	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.00	0.00	0.00	0.00	0.00	0.0	0.0	0.0
8	0.0	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.00	0.00	0.00	0.00	0.0	0.0	0.0
9	0.0	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.00	0.00	0.00	0.0	0.0	0.0
10	0.0	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.00	0.00	0.0	0.0	0.0
11	0.0	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.00	0.0	0.0	0.0
12	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.10	0.0	0.0	0.0
13	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.20	0.1	0.0	0.0
14	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.40	0.2	0.1	0.0
15	0.0	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.05	0.00	0.00	0.25	0.4	0.2	0.1

The long-run probabilities for each of the possible inventory states in policy $(15, 5)$ are presented in Figure 12. In comparison with the long-run probabilities generated for the $(14, 9)$ policy (See Figure 6), these probabilities are generally more spread among a greater number of possible inventory levels. Thus, inventory is not always forced to be higher than it must be with more ordering than necessary, which keeps both inventory and ordering costs to a minimum. Again, there is no chance inventory can become negative, as shown by the zero-probability associated with an inventory level of -1, guaranteeing no special orders will need to be made.

Figure 12: *Long-run probabilities for policy* $(15, 5)$

-1	0.000000
0	0.004590
1	0.004663
2	0.004360
3	0.027759
4	0.064623
5	0.081523
6	0.091809
7	0.093270
8	0.087190
9	0.096129
10	0.091634
11	0.081105
12	0.101318
13	0.102890
14	0.046301
15	0.020835

The expected costs for each inventory amount when using policy $(15, 5)$ are shown in Figure 13. This demonstrates the low cost of inventory compared to higher cost of orders. Every state above r, where no order is taking place, has an expected cost below \$100. Then once an order is placed the cost increases to at least \$160,000. This policy increases the number of weeks spent with these lower costs by utilizing existing inventory. It is also noteworthy that the inventory of -1 has a significant increase of almost \$40,000 compared to the inventory of 0. However, this inventory has a probability of 0, as shown in Figure 12. This demonstrates the high special-order cost, as -1 is the first inventory that requires a special order, but this high cost is impossible to reach according to the model due to r.

Figure 13: *Expected costs for each inventory amount for policy* $(15, 5)$

```
{-1: 282500, 0: 243500, 1: 227505.38461538462, 2: 211510.76923076922, 3: 195516.15384615384, 4: 179521.53846153847, 5: 163526.92307692306, 6: 32.3
07692307692314, 7: 37.69230769230769, 8: 43.07692307692308, 9: 48.46153846153847, 10: 53.846153846153854, 11: 59.23076923076924, 12: 64.6153846153
8463, 13: 70.00000000000001, 14: 75.38461538461539, 15: 80.76923076923077}
```

Overall, the $(15, 5)$ policy takes advantage of low inventory costs and avoids the high special-order costs, which add on $23000 per car that is specially ordered to the baseline ordering cost. This is the main benefit to the $(15, 5)$ policy – it only makes orders when *absolutely necessary*. Because maximum possible demand for a week is 6, an r value of 5 will prevent those special orders from occurring, and customers can count on Baxter to have available inventory. Expanding q to 15 essentially makes an order more "worthwhile", adding one more car to an order while the flat $3500 order cost remains the same. Order size may be larger, but orders are made less often compared to $(14, 9)$. Thus, customer trust is retained, but more efficiency is introduced in addition to the lowering of costs by $567 per week from $34,071.03 to $33,503.97.

6. Recommendations and Conclusion

Technical Report

This report has analyzed Baxter Volkswagen's current inventory policy and has explored all other possibilities in the effort to improve weekly decision-making policies. This was accomplished by first outlining the problem statement and assumptions required to make a Markov Chain model, then describing the development and implementation of the model in the Python programming language, and lastly analyzing and interpreting the results. Together, these steps should provide a structured and data-driven framework for evaluating inventory decisions, and we hope Baxter will find these results valuable for policy improvements going forward.

The results of the model show that to lower costs, Baxter Volkswagen should prioritize filling available inventory up to 15 vehicles when an order is placed. Orders should be infrequent to minimize the effect of the flat cost of each order. Also, special orders should be avoided as much as possible due to the relatively large penalties associated with special-ordering and losing business. This results in lowest estimated cost policy of $(15, 5)$, which accomplishes all these goals. In switching to this policy, Baxter Volkswagen decreases from a cost of $34,071.03 to $33,503.97 and saves 1.66% of the current cost. Baxter could expect to save around $567 per week in inventory and ordering costs, which is equivalent to $29,484 per year. Thus, it is recommended to switch policies, ordering up to 15 vehicles only when the inventory level reaches 5 vehicles. This will not only minimize costs but also streamline weekly business operations. The optimal policy will still ensure Baxter will retain the trust of its customers, always having necessary inventory on hand to fulfill demand.

As with all models, the result is only as good as the options that are available to Baxter to carry out its business operations. This model assumed that the only source of cars coming into the system was through an order. To further develop this model, Deloitte Consulting could integrate information on trade-ins for buying and selling used cars. This could consist of differing purchase and sale prices depending on the year the car was manufactured and milage. This could also consist of differing repair costs if the car has a history of accidents. Furthermore, if the option to incorporate used cars into inventory becomes available, this could potentially lower inventory costs, as it would be assumed the value of the car would be lower. If Baxter Volkswagen determines this could be useful information, we would be happy to work with Baxter again.

Thank you for choosing Deloitte Consulting, and we look forward to the possibility of working with you again in the future.

Nathan Schmitz and Natalie McNamara, on behalf of Deloitte Consulting

7. Appendix

Technical Report

Figure 14: *Full Python implementation of Markov Chain model*

```python
import numpy as np
import pandas as pd

# Constants
demand_probabilities = [(0, 0.1), (1, 0.2), (2, 0.4), (3, 0.25), (6, 0.05)] # List of tuples for the demand for x cars is probabili
sale_price = 20000
purchase_price = 16000
order_cost = 3500
inventory_rate = 0.0175 / 52 # Yearly cost of capital divided by weeks per year
special_order_cost = 23000 # $20000 penalty for loss in buisness plus the $3000 extra special order expences

# Get the expected value for each pair of q and r
policy_matrix = pd.DataFrame(columns=range(16), index=range(16))
for q in range(16):
    for r in range(q+1):

        # Blank transition matrix for the probaiblities of going from state to state where the rows are t-1 and columns are t
        transition_matrix = pd.DataFrame(columns=range(r-6, q+1), index=range(r-6, q+1), dtype=float)
        transition_matrix = transition_matrix.fillna(0)
        # Blank expected costs for each state
        expected_cost = dict()

        # Update the transition matrix rows and calculate costs per state
        for i in range(r-6, q+1):
            # The cost of a standard batch order when we have inventory i
            weekly_order_cost = order_cost + (q - i) * purchase_price if i <= r and i != q else 0

            # The cost of a special order when we have inventory i
            weekly_special_order_cost = special_order_cost * -i if i < 0 else 0

            # The cost of inventory when we have i cars stored
            inventory_cost = i * purchase_price * inventory_rate if i > 0 else 0

            # The total expected cost when we have i cars in inventory
            expected_cost[i] = weekly_order_cost  + weekly_special_order_cost + inventory_cost

            for cars_sold, prob in demand_probabilities:
                # The state we are in is less than r, so we ordering up to q then selling cars
                if i <= r:
                    transition_matrix.loc[i, q - cars_sold] += prob

                # The state we are in is greater than r, so we are selling from the current inventory
                else:
                    transition_matrix.loc[i, i - cars_sold] += prob

        # Use the transition matrix to get the probability of being in each state
        identity = pd.DataFrame(data=np.identity(q-r+7), columns=range(r-6, q+1), index=range(r-6, q+1))
        s = transition_matrix - identity
        s.loc[r-6:q, q] = 1
        state_probabilities = pd.Series(np.linalg.inv(s)[-1], index=range(r-6, q+1))

        # Use the cost and probability in each state to get the total expected value for this q and r policy
        policy_matrix.loc[q, r] = 0
        for i in range(r-6, q+1):
            policy_matrix.loc[q, r] += expected_cost[i] * state_probabilities[i]

#Display the minimum policy
min_q, min_r = policy_matrix.stack().idxmin()
print(f"Minimum cost policy occurs at q = {min_q}, r = {min_r}, cost = {policy_matrix.loc[min_q, min_r]:.2f}")

# Write the results to an excel file
policy_matrix.to_excel("Baxter_Auto_q_r_analysis_results.xlsx")
```